

# WSTĘP I CEL PROJEKTU

Celem projektu było zaprojektowanie i zaimplementowanie gry sieciowej "Statki" w języku Java. Projekt miał na celu praktyczne wykorzystanie wiedzy z zakresu programowania obiektowego, obsługi zdarzeń w interfejsie graficznym (GUI) oraz komunikacji sieciowej przy użyciu gniazd (Sockets). Aplikacja "**Cat Wars**" pozwala na rozgrywkę dwóch graczy w czasie rzeczywistym. Program został napisany z wykorzystaniem biblioteki **JavaFX** do obsługi warstwy wizualnej oraz mechanizmu wielowątkowości do obsługi połączenia sieciowego bez blokowania interfejsu.

## ZAŁOŻENIA I WYMAGANIA

Aplikacja została stworzona zgodnie z następującymi założeniami:

### Wymagania Funkcjonalne

- Rozgrywka Sieciowa: Możliwość połączenia dwóch instancji programu (Host i Klient) i wymiana danych o strzałach.
- Konfiguracja Planszy: Gracz ma możliwość ręcznego rozstawiania statków na planszy 10x10. System musi walidować poprawność ustawienia (statki nie mogą się stykać).
- Profile Graczy: System zapamiętuje statystyki (liczba wygranych/przegranych) i zapisuje je trwale na dysku.
- Logika Gry: Automatyczne wykrywanie trafień, zatopień i końca gry.

### Wymagania Techniczne

- Język: Java 17 lub nowsze.
- Interfejs: JavaFX (zamiast Swinga, dla nowocześniejszego wyglądu).

- Sieć: Protokół TCP/IP (zapewniający dostarczenie pakietów w kolejności).
- Architektura: Podział na logikę (Model), widok (View) i sterowanie (Controller).

## **STRUKTURA I ARCHITEKTURA SYSTEMU**

Projekt został podzielony na pakiety zgodnie ze wzorcem architektonicznym MVC (Model-View-Controller), co ułatwia zarządzanie kodem i jego rozwój.

### **Model**

Pakiety: `com.battleship.game`, `com.battleship.board`, `com.battleship.data`. Tutaj znajduje się cała matematyka gry, niezależna od tego, jak gra wygląda.

- Klasa Board: Reprezentuje planszę jako tablicę dwuwymiarową. Odpowiada za sprawdzanie kolizji statków.
- Klasa Ship: Przechowuje informacje o stanie statku (ile masztów zostało trafionych).
- Klasa Game: Przechowuje stan całej rozgrywki (np. czyja jest teraz tura).

### **Widok (Interfejs Użytkownika)**

Pakiet: `com.battleship.ui`. Odpowiada za to, co widzi użytkownik.

- Klasa FXUI: Główna klasa dziedzicząca po `Application`. Rysuje okna, siatki i obsługuje kliknięcia myszą.
- Odświeżanie: Zastosowano metodę `gameTick()`, która cyklicznie aktualizuje wygląd planszy na podstawie danych z Modelu.

## Kontroler i Sieć

Pakiety: com.battleship.controller, com.battleship.network.

- NetworkController: Jest to "most" między grą a siecią. Nasłuchuje wiadomości od drugiego gracza.
- Client: Klasa działająca na osobnym wątku (Thread). Odpowiada za fizyczne przesyłanie danych przez gniazdo (Socket).

## OPIS IMPLEMENTACJI (ROZWIĄZANIA TECHNICZNE)

W projekcie zastosowano kilka kluczowych rozwiązań programistycznych wymaganych do poprawnego działania gry sieciowej.

### Algorytm Walidacji Statków (canPlace)

Aby zapobiec nieprawidłowemu ustawieniu statków, zaimplementowano algorytm sprawdzający tzw. sąsiedztwo punktu.

- Dla każdego segmentu statku sprawdzane jest 8 pól dookoła niego.
- Jeśli w którymkolwiek sąsiednim polu znajduje się inny statek, metoda zwraca false.
- W interfejsie skutkuje to zmianą koloru "ducha" statku na czerwony.

### Komunikacja Sieciowa i Serializacja

Do przesyłania danych wykorzystano wbudowany w Javę mechanizm serializacji.

- Dane przesyłane są jako obiekty klasy Message (zawierającej typ komunikatu i dane).

- Dzięki interfejsowi Serializable, obiekty są zamieniane na ciąg bajtów, wysyłane przez strumień ObjectOutputStream i odtwarzane u drugiego gracza.

## **Wielowątkowość i Synchronizacja z UI**

Największym wyzwaniem było połączenie sieci (która czeka na dane i blokuje wątek) z interfejsem graficznym (który musi być płynny).

- Rozwiązanie: Sieć działa na osobnym wątku w tle.
- Gdy przyjdzie wiadomość (np. "Strzał w pole B5"), wątek sieciowy nie może bezpośrednio narysować X na planszy (spowodowałoby to błąd).
- Zamiast tego użyto Platform.runLater(), co bezpiecznie zleca aktualizację wyglądu do głównego wątku JavaFX.

## **Obsługa Błędów i Danych**

Zaimplementowano mechanizm "Fail-Safe" w klasie ProfileManager.

- Gra zapisuje statystyki do pliku binarnego profiles.dat.
- Przy uruchomieniu, jeśli plik jest uszkodzony, program nie wyłącza się, ale automatycznie tworzy kopię zapasową starego pliku (.bak) i generuje nową, czystą bazę danych.

# **INSTRUKCJA PROGRAMU**

Aplikacja "Cat Wars" jest systemem rozproszonym, wymagającym do działania dwóch instancji programu połączonych siecią TCP/IP. Poniższa instrukcja opisuje pełny cykl życia sesji gry: od nawiązania połączenia (Handshake), poprzez fazę strategiczną (Placement), aż po fazę aktywną (Battle) i zamknięcie sesji.

## Wymagania Wstępne i Konfiguracja Sieci

Przed przystąpieniem do gry należy upewnić się, że oba komputery znajdują się w tej samej podsieci (np. podłączone do tego samego routera Wi-Fi) lub posiadają publiczne adresy IP.

- **Port komunikacyjny:** Aplikacja domyślnie nasłuchuje na porcie 54321. Należy upewnić się, że zapor systemowa (Windows Defender/Firewall) nie blokuje ruchu przychodzącego dla aplikacji Java.
- **Adresacja:** Gracz pełniący rolę Hosta musi znać swój adres IP (można go sprawdzić w terminalu poleceniem ipconfig dla Windows lub ip a dla Linux).

### Etap 1: Inicjalizacja Połączenia (Role Sieciowe)

Gracze muszą uzgodnić podział ról przed uruchomieniem gry.

#### SCENARIUSZ DLA GRACZA A (HOST - SERWER):

1. Uruchom aplikację i wybierz swój profil gracza.
2. W Menu Głównym zlokalizuj sekcję sieciową.
3. Kliknij przycisk "STWÓRZ SESJĘ".
  - Działanie systemu: Aplikacja uruchamia ServerSocket i przechodzi w tryb aktywnego nasłuchiwanie.
4. Gracz zostaje automatycznie przeniesiony do Edytora Planszy. W tym momencie gra oczekuje na połączenie drugiego gracza w tle.

#### SCENARIUSZ DLA GRACZA B (GOŚĆ - KLIENT):

1. Uruchom aplikację i wybierz swój profil.
2. W polu tekstowym "IP (dla gościa)" wprowadź adres IPv4 komputera Hosta (np. 192.168.1.15).

- Uwaga: Jeżeli obie instancje gry są uruchomione na tej samej maszynie fizycznej, należy użyć adresu pętli zwrotnej: localhost lub 127.0.0.1.

### 3. Kliknij przycisk "DOŁĄCZ DO GRY".

- Działanie systemu: Aplikacja próbuje nawiązać połączenie TCP. W przypadku braku odpowiedzi w ciągu 5000ms (timeout), zostanie wyświetlony komunikat o błędzie połączenia.

4. Po udanym nawiązaniu połączenia (Handshake), gracz zostaje przeniesiony do Edytora Planszy.

## **Etap 2: Strategiczne Rozmieszczanie Floty (Placement Phase)**

Po nawiązaniu połączenia, obaj gracze niezależnie od siebie przygotowują swoje plansze. Jest to faza asynchroniczna – gracze nie widzą swoich ruchów.

### **Interfejs Edytora:**

- Centralna siatka: Reprezentuje obszar morski gracza (10x10 pól).
- Panel boczny (Prawy): Zawiera listę dostępnych zasobów (statków). Liczby w nawiasach [] oznaczają ilość dostępnych jednostek danego typu (np. [4] dla Kociaka).

### **Instrukcja rozmieszczania:**

1. Wybór jednostki: Kliknij Lewym Przyciskiem Myszy (LPM) na przycisk z nazwą statku (np. "Średni Kot"). Przycisk zmieni stan na aktywny (podświetlenie).

2. Pozycjonowanie i Walidacja: Przesuń kursor myszy nad siatkę gry. System w czasie rzeczywistym waliduje pozycję:

- Podświetlenie MIĘTOWE (Zielone): Pozycja jest legalna. Statek mieści się w granicach i zachowuje wymagany odstęp (1 kratka) od innych jednostek.
- Podświetlenie CZERWONE: Pozycja jest nielegalna (kolizja lub wyjście poza obszar). Kliknięcie w tym stanie nie postawi statku.

3. Rotacja: Aby zmienić orientację statku (Pionowa ↔ Pozioma), kliknij Prawy Przycisk Myszy (PPM) w dowolnym momencie celowania.

4. Zatwierdzenie: Kliknij LPM, gdy podświetlenie jest zielone, aby trwale umieścić statek. Licznik dostępnych statków w panelu bocznym zmniejszy się.

5. Narzędzia pomocnicze:

- Przycisk "LOSOWO": Uruchamia algorytm Monte Carlo, który automatycznie i natychmiastowo rozmieszcza całą flotę w losowej konfiguracji.
- Przycisk "WYCZYŚĆ": Usuwa wszystkie statki z planszy, resetując liczniki w panelu bocznym.

**Finalizacja etapu:** Przycisk "GOTOWY DO WALKI!" pozostaje nieaktywny (szary) do momentu, aż gracz rozmieści wszystkie 10 statków. Po jego kliknięciu:

- Interfejs zostaje zablokowany.
- System wyświetla ekran oczekiwania: "Czekamy na drugiego kota...".
- Gra czeka na sygnał gotowości (READY) od drugiego gracza. Gdy obie strony są gotowe, następuje automatyczne przejście do bitwy.

### **Etap 3: Aktywna Rozgrywka (Battle Phase)**

Interfejs gry ulega zmianie, prezentując dwie plansze jednocześnie. Gra toczy się w trybie turowym.

## **Elementy interfejsu:**

1. Lewa plansza (TWÓJ TEREN): Widzisz tutaj swoje statki. Na tej planszy pojawiają się znaczniki strzałów oddanych przez przeciwnika.
2. Prawa plansza (TEREN WROGA): Początkowo pusta ("Mgła Wojny"). Tutaj wykonujesz swoje ruchy.
3. Wskaźnik statusu (Góra): Informuje o stanie tury.
  - Komunikat "TWOJA TURA!" (Kolor różowy): Interfejs jest odblokowany, możesz wykonać ruch.
  - Komunikat "CZEKAMY NA WROGA..." (Kolor szary): Interfejs jest zablokowany, oczekiwanie na pakiet sieciowy od przeciwnika.

## **Mechanika strzelania:**

1. Upewnij się, że wskaźnik pokazuje "TWOJA TURA!".
2. Najedź kursorem na wybrane pole na planszy TEREN WROGA.
3. Kliknij LPM. System wysyła pakiet z koordynatami do drugiego gracza i oczekuje na odpowiedź.
4. Reakcja systemu (Feedback):
  - TRAFIENIE: Na polu pojawia się czerwony znak "X", odtwarzany jest dźwięk eksplozji. Zgodnie z zasadami, zachowujesz turę i możesz oddać kolejny strzał.
  - PUDŁO: Na polu pojawia się niebieska kropka "●", odtwarzany jest dźwięk plusku wody. Tura natychmiast przechodzi do przeciwnika.
  - ZATOPIENIE: Jeśli trafienie zniszczyło ostatni segment statku, wyświetlany jest komunikat "TRAFIONY ZATOPIONY!", a statek przeciwnika zmienia stan na zniszczony.

**Oznaczanie taktyczne:** Gracz może użyć Prawego Przycisku Myszy (PPM) na planszy wroga, aby postawić ikonę "łapki" (  ). Jest to



znacznik lokalny, niewidoczny dla przeciwnika, służący do oznaczania pól, w których gracz przewiduje brak statków (np. pola wokół zatopionego okrętu).

## **Zakończenie Gry i Rozłączenie**

System na bieżąco monitoruje stan floty obu graczy. Warunkiem końca gry jest zatopienie wszystkich 10 statków (20 masztów) jednego z graczy.

### **1. Ekran Wyniku:**

- Zwycięzca otrzymuje ekran gratulacyjny: "MIAU! WYGRANA! 🐱" wraz z fanfarami.
- Przegrany otrzymuje informację: "UPS... PRZEGRANA 🐱" z dźwiękiem porażki.

**2. Aktualizacja Profilu:** Aplikacja automatycznie aktualizuje statystyki w pliku profiles.dat (zwiększenie licznika wygranych/przegranych, dodanie czasu gry).

### **3. Zamknięcie Sesji:**

- Kliknięcie przycisku "WRÓĆ DO MENU" powoduje bezpieczne zamknięcie gniazda sieciowego (socket.close()), zatrzymanie wątków nasłuchujących i powrót do ekranu startowego.
- Aplikacja jest gotowa do rozpoczęcia nowej sesji bez konieczności restartu.

## **AUTORZY:**

NATALIA CHRUŚCIEL 097054

KAMIL LUBIENIECKI 097128

Informatyka II Rok – Semestr zimowy – 2ID11A